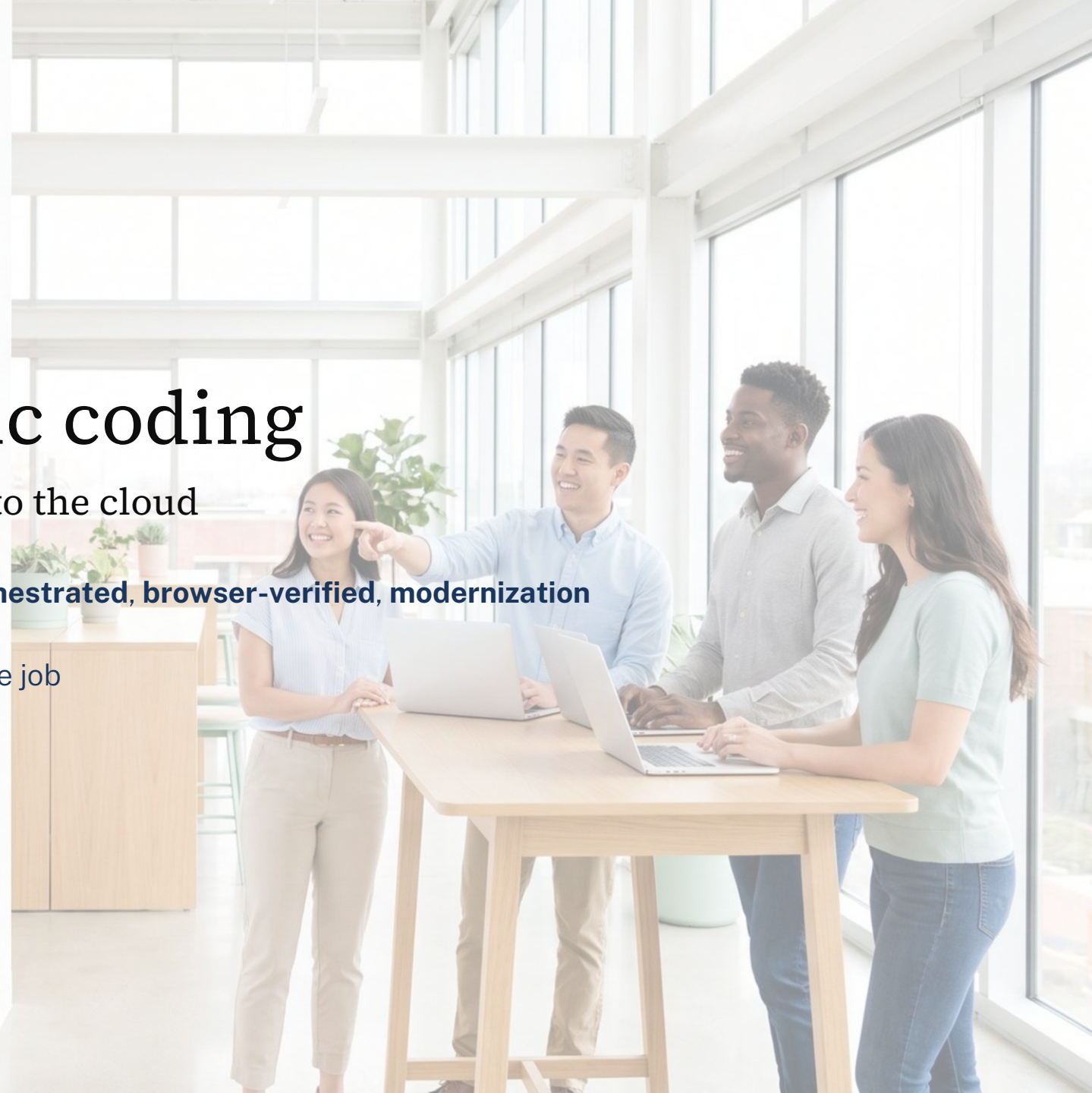


Implementing agentic coding

Putting agents to work, from your laptop to the cloud

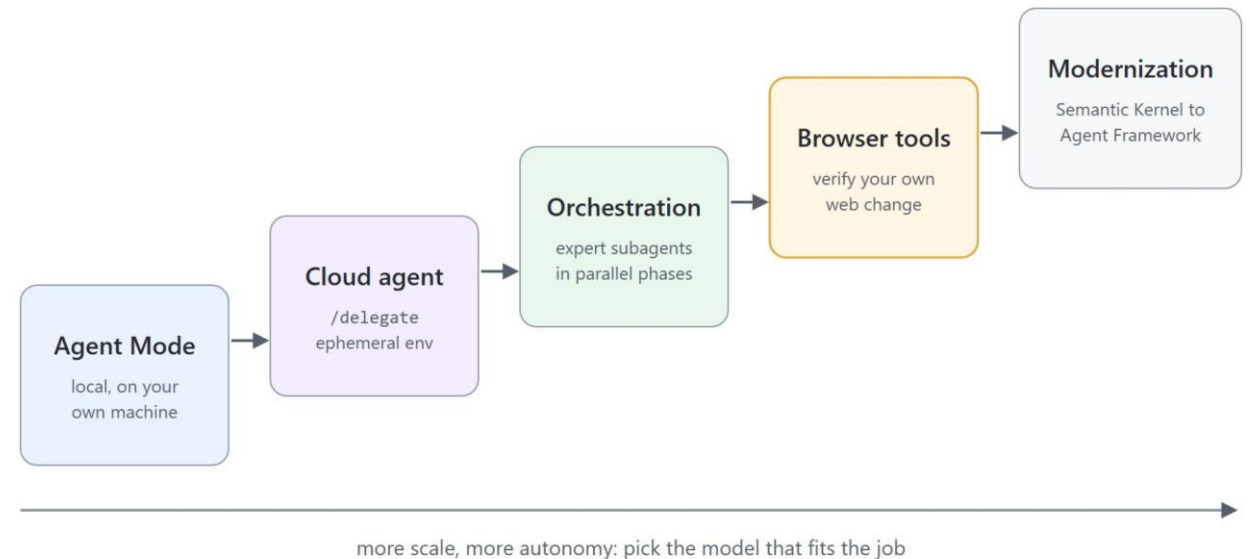
- Five ways to put an agent to work: **local, cloud, orchestrated, browser-verified, modernization**
- Start small in Agent Mode, then scale out
- Leave able to pick the right delegation model for the job



Five ways to put agents to work

From one local loop to a team and a migration

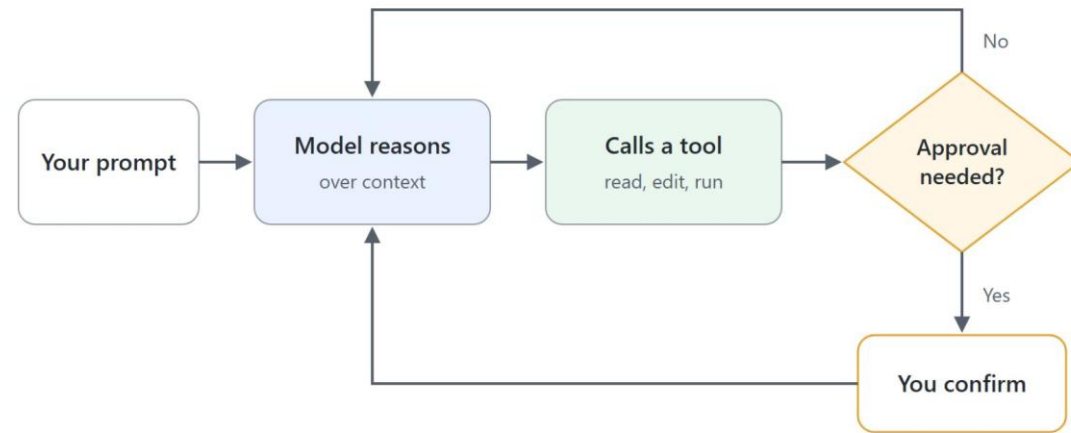
- **Agent Mode:** the baseline, on your machine with your tools and auth
- **Cloud agents:** heavy jobs in an ephemeral remote environment
- **Orchestration:** expert subagents in parallel phases
- **Browser tools** and **modernization:** verify web changes, migrate frameworks



The agent loop

Reason, call a tool, observe, repeat

- An agent turn is **not one model call**: it reasons, calls a tool, reads the result, reasons again
- Tools are the **verbs**: read a file, edit a file, run a command, call an MCP server
- Some verbs change your machine, so the loop has an **approval gate**
- The gate is the part you configure

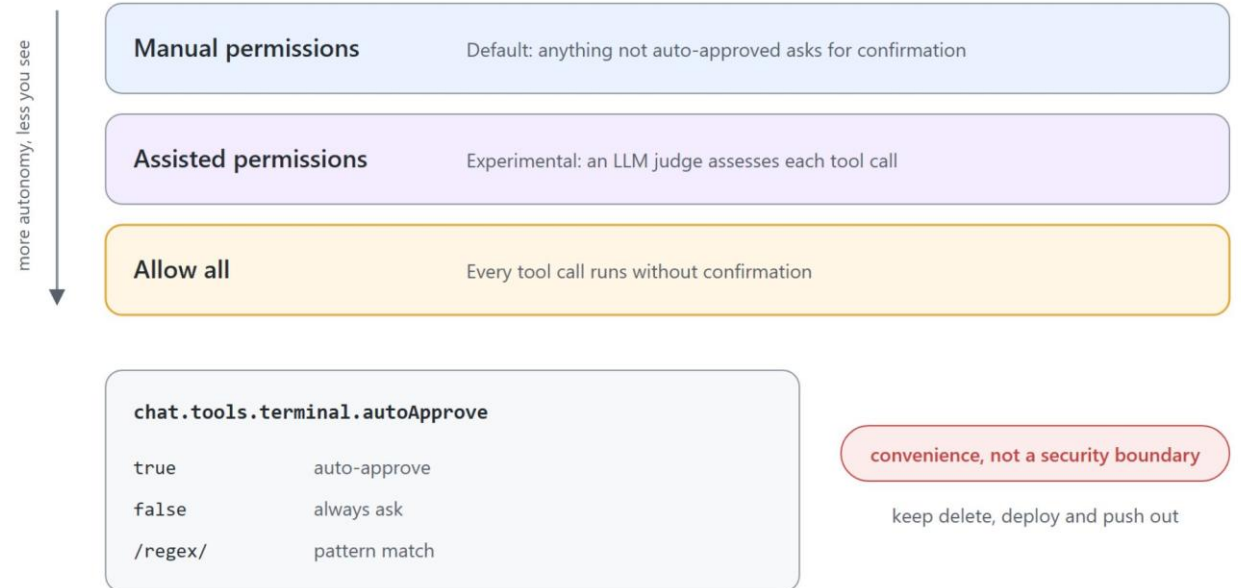


repeat until done, blocked on you, or stopped

Approvals and permission levels

Chosen per session, loosened deliberately

- **Manual permissions:** the default, anything not auto-approved asks
- **Assisted permissions:** an LLM judge assesses each tool call
- **Allow all:** every tool call runs without confirmation
- `chat.tools.terminal.autoApprove: true, false, or a /regex/;` keep delete, deploy, and push out



Steering, queueing, and checkpoints

A long run is a conversation, not a submission

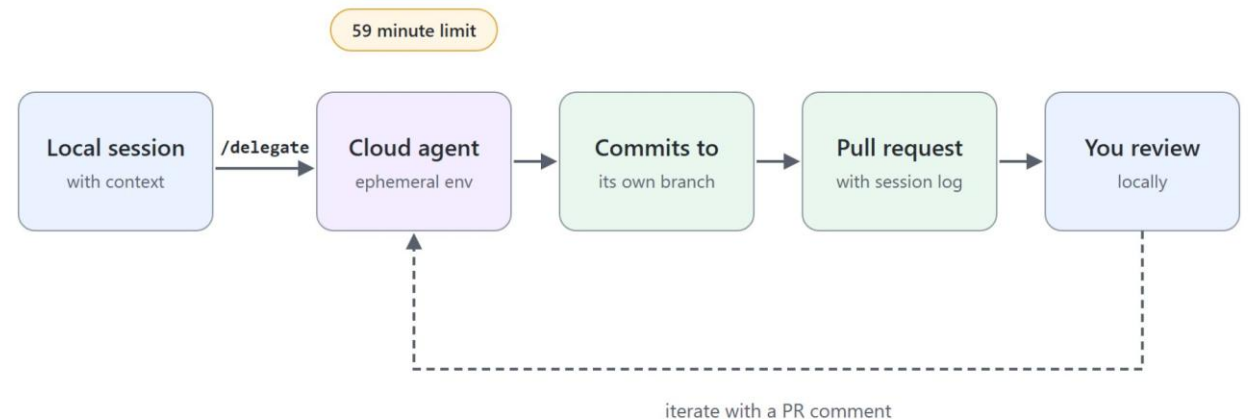
- **Steering** injects a correction into the request in flight
- **Queueing** waits for it to finish, then runs next
(`chat.requestQueueing.defaultAction`)
- A `!` prefix skips the model and runs a terminal command
- **Checkpoints** roll back workspace files and chat history together; stopping does not undo



Handing off to a cloud agent

`/delegate` carries the conversation to the cloud

- Type `/delegate` in a running session, or pick **Cloud** as the session target
- History and context travel with the task
- The agent works on **its own branch**, commits, and opens a **pull request**
- Iterate with a PR comment instead of starting over

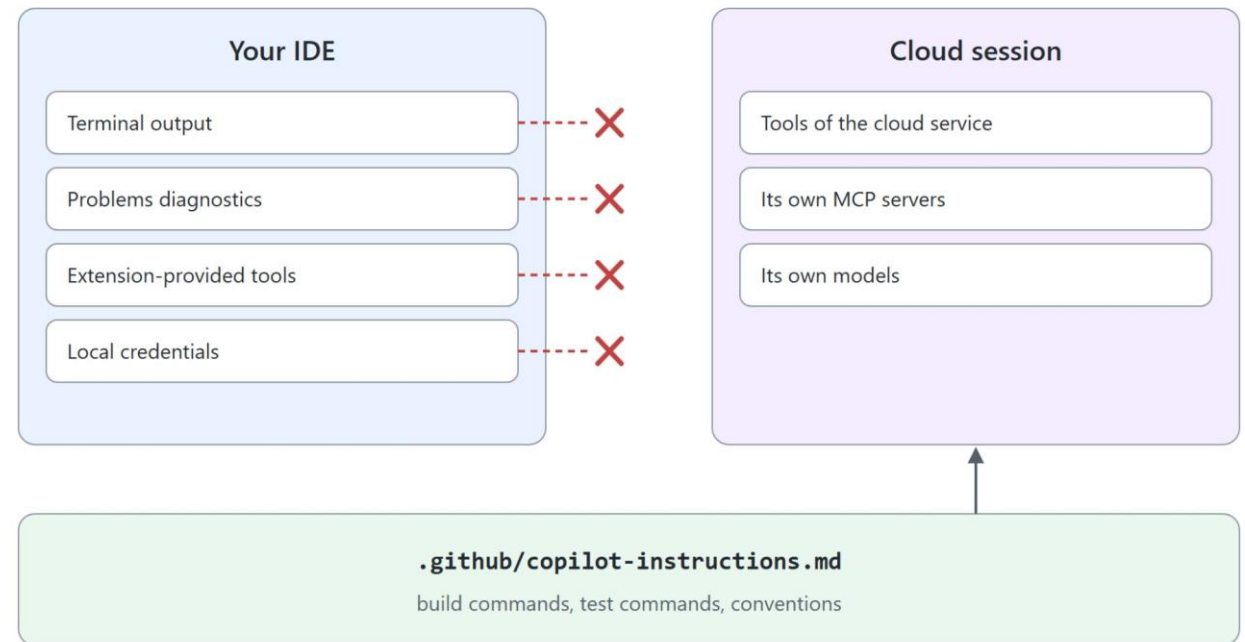


the local session stays the hub you return to

What the cloud agent cannot see

The repository has to carry the context

- Cloud sessions use the **cloud service's** tools, MCP servers, and models
- No terminal output, no Problems panel, no extension tools, **no local credentials**
- **.github/copilot-instructions.md** carries build, test, and conventions
- Most disappointing cloud runs trace back to missing context



Scoping a task worth delegating

A written brief, not a conversation

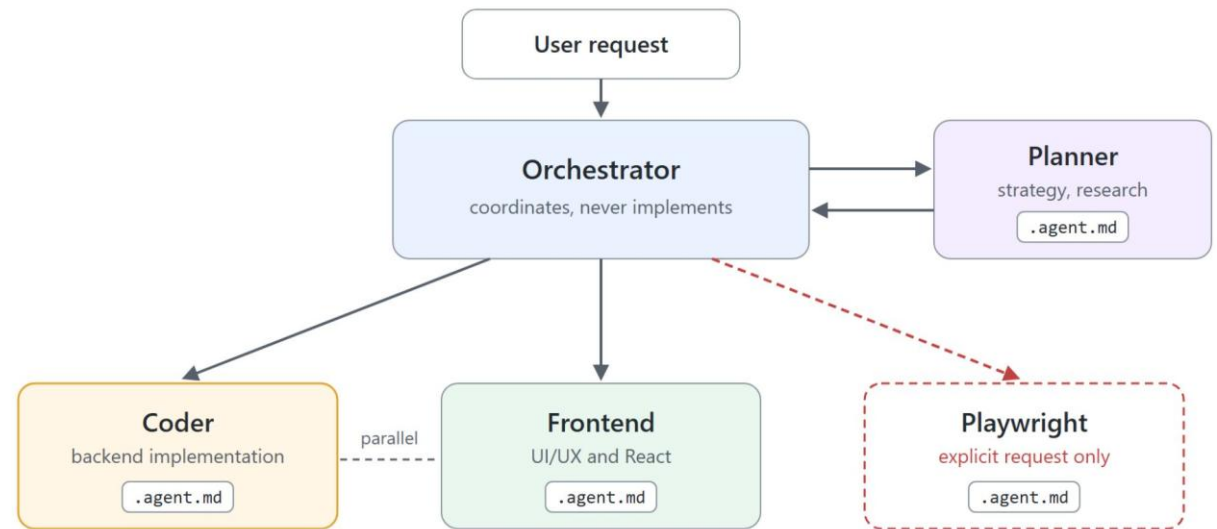
- Nobody is there to answer a mid-run question
- A good brief: **problem, acceptance criteria, which files change**
- Fits: test coverage, reproducible bugs, config and docs, mechanical refactors
- Keep exploratory or turn-by-turn work local



Subagents as subject-matter experts

One orchestrator, a team of focused experts

- A subagent works in **its own transcript**, keeping the primary context clean
- Backed by an **.agent.md**: focused scope, instructions, curated tools, right model
- **Orchestrator** delegates to **Planner**, then **Coder** and **Frontend** in parallel
- You add capacity by adding an expert, not by growing one prompt

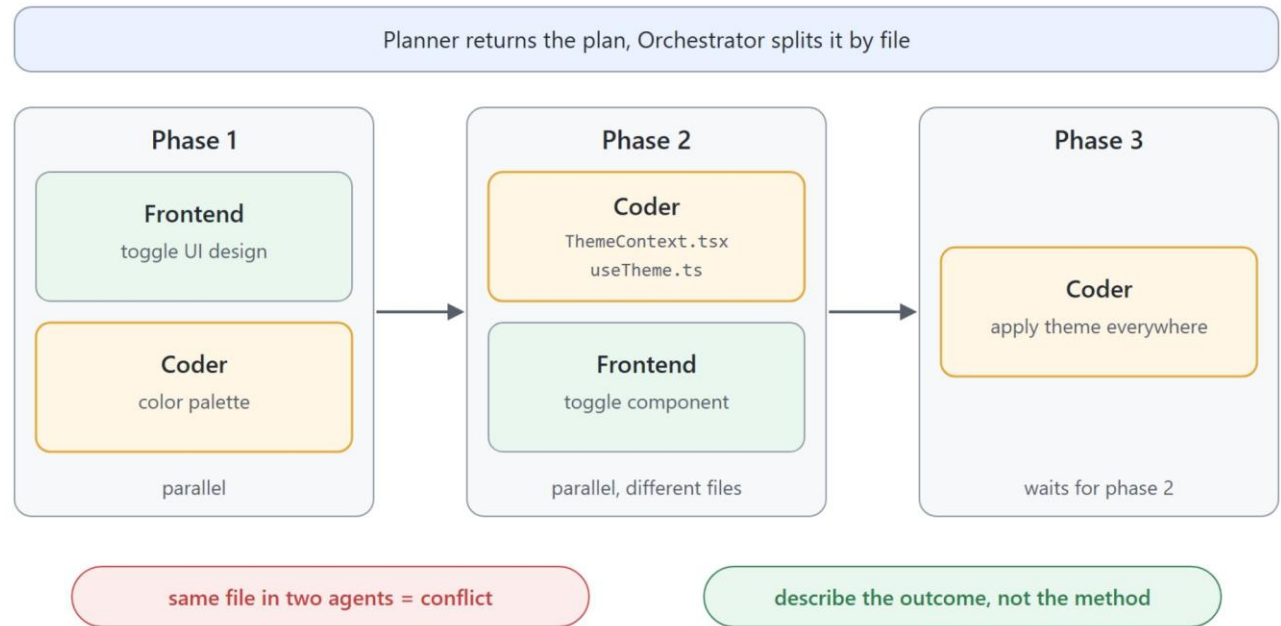


add capacity by adding an expert, not by growing one prompt

Parallel phases without file conflicts

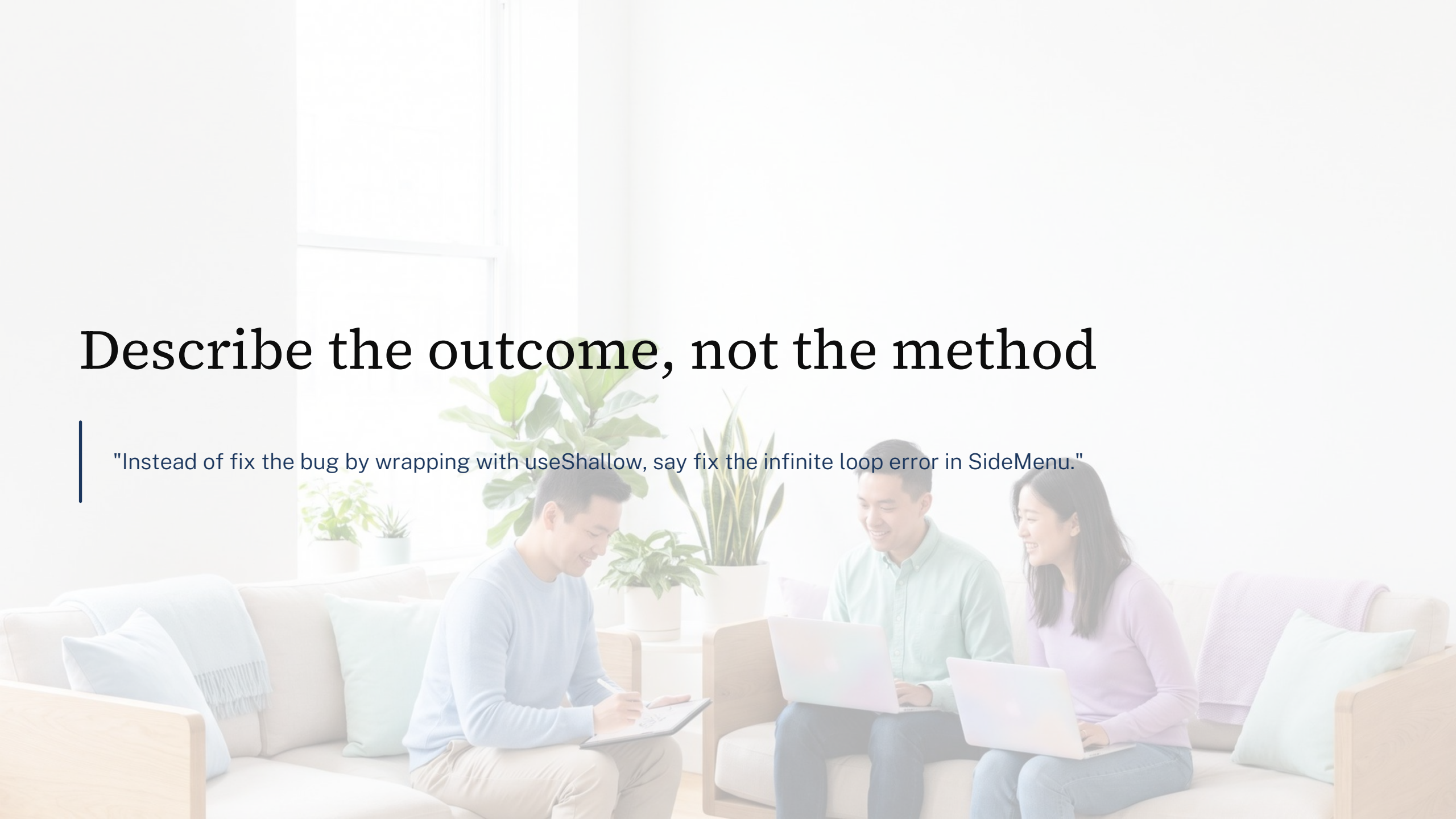
Different files run together, shared files wait

- Orchestrator: **plan, parse phases, execute, verify**
- Tasks with **no overlapping files** run in parallel
- Tasks with **shared files** run sequentially
- Scope each agent to **distinct files**



Describe the outcome, not the method

"Instead of fix the bug by wrapping with useShallow, say fix the infinite loop error in SideMenu."



Observe and account for delegated work

Read-only peer chats, live status, credits per expert

- Open a subagent's transcript as a **read-only peer chat**: follow it, never steer it
- Agents window shows **model, elapsed time, current tool call** per subagent
- Activity pills: live counts for **Changes, Previews, Subagents, Browsers**
- Each subagent shows its **own credit cost** on hover; pin an exact model ID



Agentic browser automation

Built in, on by default, no MCP server

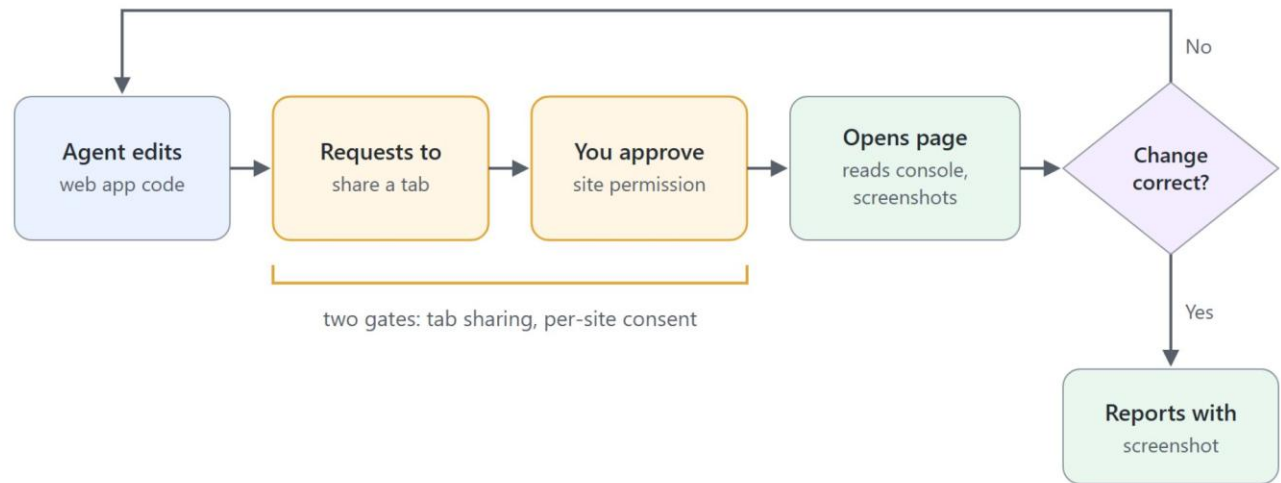
- Open and navigate, **read the console**, **screenshot into chat**, click and type
- **Device emulation** through generated Playwright code you can keep as a test
- Favorites, history, and a remote proxy for corporate networks
- Governed by the **BrowserChatTools** policy



The change-then-verify loop

Tab sharing and per-site consent keep you in the loop

- The agent **never grabs a tab**: it sends a share request, you pick the tab
- First visit to a site raises a **per-site permission** prompt
- Approving `localhost` does not grant the admin portal on another host
- Nothing is shared with the agent silently



Annotating a page for the agent

Point at the element instead of describing it

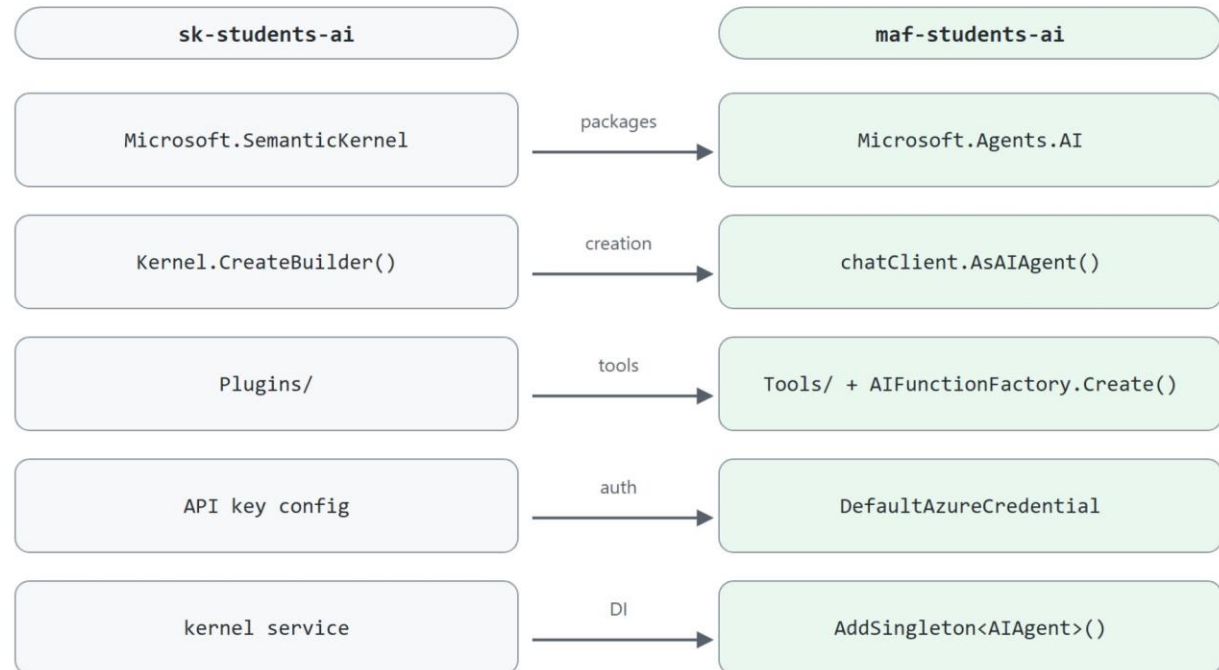
- Select an element with **Ctrl+Alt+C** (Option+Cmd+C on macOS) and comment on it
- The agent gets **the element and your note** together
- `workbench.browser.autoReloadOnFileChange` reloads local HTML on every edit
- Edit and rendered result stay in step without a manual refresh



Semantic Kernel to Agent Framework

The same RAG app, migrated by an agent

- **Packages:** `Microsoft.SemanticKernel` to `Microsoft.Agents.AI`
- **Creation:** kernel builder to `chatClient.AsIAgent()`
- **Tools:** `Plugins` to `Tools` with `AIFunctionFactory.Create()`
- **Auth and DI:** `DefaultAzureCredential`, agent as a singleton



Let the agent research the migration

Microsoft Learn MCP for versions, APIs, and breaking changes

- Ask Learn MCP for the **latest stable versions** and **breaking changes**
- Research **ChatClientAgent** before refactoring `Program.cs`
- Compare **plugins and tools** before renaming the folder
- One prompt per migration step: packages, creation, tools, auth, DI



Pick the right way to delegate

Match the job to the agent

- **Bounded and interactive:** Agent Mode
- **Well-scoped and long:** cloud agent with /delegate
- **Spans domains:** orchestrator with expert subagents
- **Web change** or **migration:** let the agent verify and research

